Université de Haute-Alsace
ENSISA

ENSISA IR 3ème année
Année 2026/2027

Programmation Rust

TP2 : Implémentation d'un morpion

Goals

Rappeller les bases de Python
Implementer un morpion
Gérer les entrées utilisateurs
Découvrir "l'objet" en Rust

Instructions

Lisez bien l'énoncé, il contient beaucoup d'informations et chaque mot est utile!

Aidez-vous de la documentation que je vous indiquerai. Lorsqu'il utilise une nouvelle librairie, un développeur passe beaucoup de temps dans la documentation pour comprendre son fonctionnement.

Codez proprement avec des noms de variables et de fonctions/méthodes bien choisis. Ajoutez des commentaires utiles à votre code afin de pouvoir encore le comprendre dans quelques jours.

Respectez le coding style de Rust[1]. `cargo fmt` peut déjà corriger certaines choses.

Simple is beautiful. N'essayez pas de coder compliqué, restez simple, cela sera plus efficace et moins sujet aux erreurs, surtout en Rust.

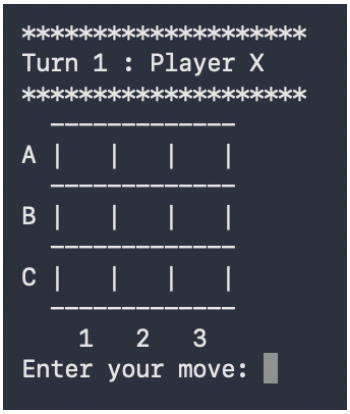
Réfléchissez avant de coder, prenez un peu de temps pour dessiner/écrire vos idées sur une feuille. Le temps de la réflexion que vous prendrez, vous économisera énormément de temps de debug.

Utiliser `cargo clippy` pour vérifier votre code.

1 Morpion basique

Nous allons réaliser un morpion simple pour ne pas perdre de temps et nous concentrer sur le Rust, je vous mets une version Python sur Moodle (qui vous rappellera des souvenirs), créez un projet cargo et re-écrivez la en Rust sans la donner bêtement à une IA...

Vous devriez obtenir visuellement ceci :



Tout est bon? Félicitations, vous avez fait votre premier vrai programme Rust ou plutot traduit votre premier programme en Rust. Mais nous pouvons faire mieux ... vous le verrez dans la section suivante!

1. Style guide de Rust : `https://doc.rust-lang.org/style-guide/`

2 Morpion version objet

Maintenant que vous avez une version de base, nous pouvons améliorer la beauté et la puissance de notre code en en faisant une version "objet. Rust n'a pas de notion de classe mais il utilise des `struct` pour les données et des `impl` pour les méthodes. Nous allons prendre notre code précédent et le mettre en version "objet", pour cela, suivez les instructions ci-dessous.

1. Créez un projet cargo `tictactoe_improved`
2. Créez une `struct` TicTacToe qui va gérer le jeu accompagnée de méthodes dans un `impl`
3. Créez une `struct` Board qui représentera le plateau et le gérera accompagnée de méthodes dans un `impl`
4. Incluez votre code précédent dans ces deux "classes" et adaptez-le au contexte "objet".
5. Ajoutez une méthode au `impl` TicTacToe réinitialisant le jeu

3 Bonus

Vous avez tout fini? Félicitations, maintenant vous pouvez faire la même chose mais pour un **Puissance 4**.